Go (Golang) Functions

In **Go (Golang)**, functions are first-class citizens. They are simple, powerful, and support advanced concepts like multiple return values, named returns, variadic parameters, and closures.

1️⃣ Basic Function Syntax

```
package main

import "fmt"


// function definition

func greet() {

   fmt.Println("Hello, Go!")

}


func main() {

   greet() // function call

}
```

Structure:

```
func functionName(parameters) returnType {

   // body

}
```

2️⃣ Function With Parameters

```
func greet(name string) {
   fmt.Println("Hello", name)
}

func main() {
   greet("Dhivya")
}
```

3️⃣ Function With Return Value

```
func add(a int, b int) int {
   return a + b
}

func main() {
   result := add(10, 5)
   fmt.Println(result)
}
```

Shortened Parameter Type

```
func add(a, b int) int {
   return a + b
}
```

4️⃣ Multiple Return Values (Very Important in Go)

Go supports returning multiple values.

```
func divide(a, b int) (int, int) {
   quotient := a / b
   remainder := a % b
   return quotient, remainder
}

func main() {
   q, r := divide(10, 3)
   fmt.Println("Quotient:", q)
   fmt.Println("Remainder:", r)
}
```

Common Pattern: Return Value + Error

```
func safeDivide(a, b int) (int, error) {
   if b == 0 {
      return 0, fmt.Errorf("cannot divide by zero")
   }
   return a / b, nil
}
```

5️⃣ Named Return Values

```
func calculate(a, b int) (sum int, product int) {
   sum = a + b
```

```
    product = a * b
    return
}
```

6️⃣ Variadic Functions

Used when number of arguments is unknown.

```
func sum(numbers ...int) int {
    total := 0
    for _, num := range numbers {
        total += num
    }
    return total
}

func main() {
    fmt.Println(sum(1, 2, 3, 4))
}
```

7️⃣ Anonymous Functions

Functions without a name.

```
func main() {
    greet := func(name string) {
        fmt.Println("Hello", name)
```

```
    }

    greet("Dhivya")
}
```

8️⃣ Immediately Invoked Function (IIFE)

```
func main() {
    func() {
        fmt.Println("Executed immediately")
    }()
}
```

9️⃣ Functions as Parameters (Higher-Order Functions)

```
func operate(a int, b int, operation func(int, int) int) int {
    return operation(a, b)
}

func main() {
    result := operate(10, 5, func(x, y int) int {
        return x + y
    })
    fmt.Println(result)
}
```

🔟 Closures

A closure remembers variables from outer scope.

```
func counter() func() int {
  count := 0
  return func() int {
    count++
    return count
  }
}

func main() {
  c := counter()
  fmt.Println(c()) // 1
  fmt.Println(c()) // 2
}
```

1️⃣1️⃣ Recursive Functions

```
func factorial(n int) int {
  if n == 0 {
    return 1
  }
  return n * factorial(n-1)
```

```
}
```

1️⃣2️⃣ Methods (Functions With Receiver)

Go doesn't have classes, but it has methods.

```
type Person struct {
   name string
}


func (p Person) greet() {
   fmt.Println("Hello", p.name)
}
```

1️⃣3️⃣ Exported vs Unexported Functions

In Go:

- Capital letter → Exported (public)
- Small letter → Unexported (private)

```
func Add(a, b int) int {  // exported
   return a + b
}


func subtract(a, b int) int { // private
   return a - b
}
```